



Instituto Politécnico Nacional



Unidad Profesional Interdisciplinaria en Ingeniería y
Tecnologías Avanzadas

Ingeniería Telemática

Base de Datos Distribuidos

2TM3

Carlos De La Cruz Sosa

Propuesta de Fragmentación

Integrantes:

García Ortiz Martha Lesly

Hernández Arteaga Itzel

25 de septiembre de 2026

Objetivo

Distribuir la cartera de clientes en los 5 nodos geográficos para que cada región administre y consulte de forma autónoma los datos de sus propios compradores. Tabla Sales.Customer, columna CustomerId (rangos).

Segmento de modelo relacional

De acuerdo con el grafo del modelo relacional mostrado en la Figura 1, la tabla Sales.Customer mantiene relaciones con diversas tablas. Sin embargo, para este análisis se seleccionaron Sales.SalesOrderHeader y Sales.SalesOrderDetail, debido a que permiten conservar la relación entre los clientes, sus órdenes de compra y los detalles de dichas órdenes.

La tabla Sales.Customer se relaciona directamente con Sales.SalesOrderHeader mediante el atributo CustomerID, el cual es llave primaria en Customer y llave foránea en SalesOrderHeader. Esta relación permite identificar qué órdenes pertenecen a cada cliente y, por lo tanto, derivar la fragmentación de SalesOrderHeader a partir de los fragmentos definidos para Customer.

Posteriormente, Sales.SalesOrderHeader se relaciona con Sales.SalesOrderDetail mediante SalesOrderID, que identifica cada orden y permite determinar qué detalles corresponden a cada una. De esta manera, la fragmentación puede propagarse desde los clientes hacia sus órdenes y, finalmente, hacia los detalles de dichas órdenes.

Por lo tanto, se seleccionaron estas dos tablas como relaciones miembro para la fragmentación derivada, ya que permiten mantener en cada nodo no solamente los clientes correspondientes a su rango, sino también las órdenes realizadas por esos clientes y los detalles asociados a dichas órdenes.

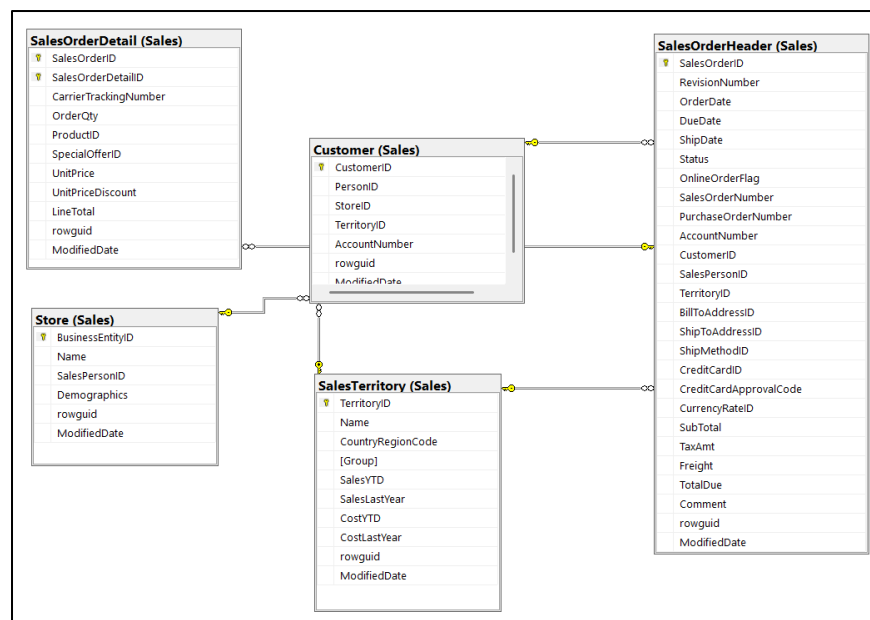


Fig.1 Modelo relacional

Algoritmo COM_MIN

Para aplicar COM_MIN se partió de la tabla propietaria Sales.Customer, la cual fue seleccionada para realizar la fragmentación horizontal primaria. El atributo utilizado para la fragmentación fue CustomerID, ya que este permite dividir la cartera de clientes en cinco fragmentos correspondientes a los nodos regionales: Norte, Sur, Este, Oeste y Centro.

El objetivo de aplicar el algoritmo COM_MIN es obtener un conjunto de predicados simples que sea completo y mínimo para realizar la fragmentación horizontal primaria de la tabla Sales.Customer.

El algoritmo COM_MIN permite verificar que los predicados propuestos sean necesarios para generar los fragmentos y que no existan predicados redundantes. De esta manera, se busca que el diseño de fragmentación sea claro, completo y sin condiciones innecesarias.

Primero se realizó un análisis del atributo CustomerID para conocer su valor mínimo, su valor máximo y la cantidad total de registros existentes en la tabla. Para ello se ejecutó la siguiente consulta:

```
SELECT  
MIN(CustomerID) AS MinimoID,  
MAX(CustomerID) AS MaximoID,  
COUNT(*) AS TotalClientes  
FROM Sales.Customer;
```

El resultado obtenido fue:

MinimoID = 1

MaximoID = 30118

TotalClientes = 19820

Como el ejercicio solicita distribuir la cartera de clientes en cinco regiones, se dividió el dominio del atributo CustomerID en cinco rangos. Para ello se calculó el tamaño aproximado de cada rango utilizando el valor mínimo y máximo del atributo.

Tamaño del rango = $(\text{MaximoID} - \text{MinimoID} + 1) / 5$

Sustituyendo los valores obtenidos:

Tamaño del rango = $(30118 - 1 + 1) / 5$

Tamaño del rango = $30118 / 5$

Tamaño del rango = 6023.6

L1 = 6024

L2 = 12048

L3 = 18072

L4 = 24096

Con base en estos límites, se definió el conjunto de predicados simples. Un predicado simple tiene esta forma: atributo | operador | valor |

Entonces los predicados tendrán esta forma:

CustomerID <= límite_1

CustomerID <= límite_2

CustomerID <= límite_3

CustomerID <= límite_4

¿Por qué solo 4 límites si queremos 5 regiones?

Porque 4 cortes dividen un rango en 5 partes.

Ejemplo visual:

-----	-----	-----	-----	-----
Norte	Sur	Este	Oeste	Centro
L1	L2	L3	L4	

PR = {
p1: CustomerID <= 6024,
p2: CustomerID <= 12048,
p3: CustomerID <= 18072,
p4: CustomerID <= 24096
}

Obtención de los miniterminos

Una vez definido el conjunto de predicados simples:

PR={p1,p2,p3,p4}

donde:

p1: CustomerID <= 6024,
p2: CustomerID <= 12048,
p3: CustomerID <= 18072,
p4: CustomerID <= 24096

se aplica el algoritmo COM_MIN para verificar que el conjunto de predicados sea completo y mínimo con respecto a los cinco rangos de acceso propuestos.

En este caso, los cuatro predicados son necesarios, ya que cada uno representa un punto de corte entre dos regiones. Si se eliminara alguno de ellos, dos rangos consecutivos no podrían

distinguirse. Por lo tanto, no se identifican predicados redundantes y el conjunto mínimo queda:

$$\{PR'=PR=\{p_1,p_2,p_3,p_4\}\}$$

A partir de este conjunto se construyen los minitérminos, utilizando los predicados y sus negaciones. Un minitérmino representa una combinación lógica de predicados que permite identificar de manera única cada uno de los fragmentos.

Para la primera región se tiene:

$$m_1=p_1$$

Sustituyendo p_1 :

$$m_1: CustomerID \leq 6024$$

Por lo tanto, m_1 corresponde al fragmento **Norte**.

Para la segunda región:

$$m_2 = \neg p_1 \wedge p_2$$

donde el símbolo ($\neg$) significa **negación** y ($\wedge$) significa **AND (y)**.

Como:

$$\neg p_1: CustomerID > 6024$$

y:

$$p_2: CustomerID \leq 12048$$

entonces:

$$m_2: 6024 < CustomerID \leq 12048$$

Este minitérmino corresponde al fragmento **Sur**.

Siguiendo el mismo procedimiento, los cinco minitérminos resultantes son:

Minitérmino	Expresión lógica	Condición resultante	Región
m_1	p_1	$CustomerID \leq 6024$	Norte
m_2	$\neg p_1 \wedge p_2$	$6024 < CustomerID \leq 12048$	Sur
m_3	$\neg p_2 \wedge p_3$	$12048 < CustomerID \leq 18072$	Este
m_4	$\neg p_3 \wedge p_4$	$18072 < CustomerID \leq 24096$	Oeste
m_5	$\neg p_4$	$CustomerID > 24096$	Centro

De esta manera, el conjunto de minitérminos puede expresarse como:

$$M = \{m_1, m_2, m_3, m_4, m_5\}$$

donde:

$$M = \{p_1, \neg p_1 \wedge p_2, \neg p_2 \wedge p_3, \neg p_3 \wedge p_4, \neg p_4\}$$

Relación entre los minitérminos y los fragmentos

Cada minitérmino genera un fragmento horizontal de la tabla Sales.Customer. Utilizando el operador de selección σ del álgebra relacional, los fragmentos quedan definidos de la siguiente manera:

$$\begin{aligned} C_1 &= \sigma_{CustomerID \leq 6024}(Customer) \\ C_2 &= \sigma_{6024 < CustomerID \leq 12048}(Customer) \\ C_3 &= \sigma_{12048 < CustomerID \leq 18072}(Customer) \\ C_4 &= \sigma_{18072 < CustomerID \leq 24096}(Customer) \\ C_5 &= \sigma_{24096 < CustomerID \leq 30118}(Customer) \end{aligned}$$

Por lo tanto, los minitérminos obtenidos permiten generar los cinco fragmentos horizontales correspondientes a los nodos Norte, Sur, Este, Oeste y Centro.

Implementación SQL de la fragmentación primaria

/*FRAGMENTO 1: NORTE*/

```
SELECT *
FROM Sales.Customer
WHERE CustomerID >= 1 AND CustomerID <= 6024;
```

/*FRAGMENTO 2: SUR*/

```
SELECT *
FROM Sales.Customer
WHERE CustomerID > 6024 AND CustomerID <= 12048;
```

/*FRAGMENTO 3: ESTE*/

```
SELECT *
FROM Sales.Customer
WHERE CustomerID > 12048 AND CustomerID <= 18072;
```

/*FRAGMENTO 4: OESTE*/

```
SELECT *
FROM Sales.Customer
WHERE CustomerID > 18072 AND CustomerID <= 24096;
```

/*FRAGMENTO 5: CENTRO*/

```
SELECT *
FROM Sales.Customer
WHERE CustomerID > 24096 AND CustomerID <= 30118;
```

Fragmentos derivados a partir de la operación SEMI-JOIN

El objetivo de este apartado es generar los fragmentos derivados a partir de los fragmentos primarios de la tabla Sales.Customer, utilizando la operación SEMI-JOIN.

En este diseño, la tabla propietaria es Sales.Customer, la cual fue fragmentada horizontalmente en cinco regiones: Norte, Sur, Este, Oeste y Centro. A partir de estos fragmentos se deriva la tabla Sales.SalesOrderHeader, ya que esta contiene el atributo CustomerID, el cual permite relacionar cada orden de venta con el cliente correspondiente.

El propósito de la fragmentación derivada es que las órdenes de venta queden ubicadas en la misma región que el cliente al que pertenecen, manteniendo la relación lógica entre los datos de clientes y sus órdenes.

Fragmentación derivada de SalesOrderHeader

Conceptualmente usamos un **SEMI-JOIN**:

$$SOH_i = SalesOrderHeader \bowtie_{SOH.CustomerID=C_i.CustomerID} C_i$$

USE AdventureWorks;
GO

```
/* =====  
FRAGMENTACIÓN DERIVADA: SalesOrderHeader  
===== */
```

```
/* FRAGMENTO 1: NORTE */  
SELECT SOH.*  
FROM Sales.SalesOrderHeader AS SOH  
INNER JOIN Sales.Customer AS C  
    ON SOH.CustomerID = C.CustomerID  
WHERE C.CustomerID >= 1  
    AND C.CustomerID <= 6024;
```

```
/* FRAGMENTO 2: SUR */  
SELECT SOH.*  
FROM Sales.SalesOrderHeader AS SOH  
INNER JOIN Sales.Customer AS C  
    ON SOH.CustomerID = C.CustomerID  
WHERE C.CustomerID > 6024  
    AND C.CustomerID <= 12048;
```

```
/* FRAGMENTO 3: ESTE */
```

```

SELECT SOH.*
FROM Sales.SalesOrderHeader AS SOH
INNER JOIN Sales.Customer AS C
    ON SOH.CustomerID = C.CustomerID
WHERE C.CustomerID > 12048
    AND C.CustomerID <= 18072;

```

```

/* FRAGMENTO 4: OESTE */
SELECT SOH.*
FROM Sales.SalesOrderHeader AS SOH
INNER JOIN Sales.Customer AS C
    ON SOH.CustomerID = C.CustomerID
WHERE C.CustomerID > 18072
    AND C.CustomerID <= 24096;

```

```

/* FRAGMENTO 5: CENTRO */
SELECT SOH.*
FROM Sales.SalesOrderHeader AS SOH
INNER JOIN Sales.Customer AS C
    ON SOH.CustomerID = C.CustomerID
WHERE C.CustomerID > 24096
    AND C.CustomerID <= 30118;

```

Fragmentación derivada de SalesOrderDetail

Después propagamos la fragmentación un nivel más:

$$Customer_i \rightarrow SOH_i \rightarrow SOD_i$$

Conceptualmente:

$$SOD_i = SalesOrderDetail \bowtie_{SOD.SalesOrderID=SOH_i.SalesOrderID} SOH_i$$

```

/* =====
FRAGMENTACIÓN DERIVADA: SalesOrderDetail
===== */

```

```

/* FRAGMENTO 1: NORTE */
SELECT SOD.*
FROM Sales.SalesOrderDetail AS SOD
INNER JOIN Sales.SalesOrderHeader AS SOH
    ON SOD.SalesOrderID = SOH.SalesOrderID
INNER JOIN Sales.Customer AS C

```

```
ON SOH.CustomerID = C.CustomerID
WHERE C.CustomerID >= 1
AND C.CustomerID <= 6024;
```

```
/* FRAGMENTO 2: SUR */
SELECT SOD.*
FROM Sales.SalesOrderDetail AS SOD
INNER JOIN Sales.SalesOrderHeader AS SOH
    ON SOD.SalesOrderID = SOH.SalesOrderID
INNER JOIN Sales.Customer AS C
    ON SOH.CustomerID = C.CustomerID
WHERE C.CustomerID > 6024
AND C.CustomerID <= 12048;
```

```
/* FRAGMENTO 3: ESTE */
SELECT SOD.*
FROM Sales.SalesOrderDetail AS SOD
INNER JOIN Sales.SalesOrderHeader AS SOH
    ON SOD.SalesOrderID = SOH.SalesOrderID
INNER JOIN Sales.Customer AS C
    ON SOH.CustomerID = C.CustomerID
WHERE C.CustomerID > 12048
AND C.CustomerID <= 18072;
```

```
/* FRAGMENTO 4: OESTE */
SELECT SOD.*
FROM Sales.SalesOrderDetail AS SOD
INNER JOIN Sales.SalesOrderHeader AS SOH
    ON SOD.SalesOrderID = SOH.SalesOrderID
INNER JOIN Sales.Customer AS C
    ON SOH.CustomerID = C.CustomerID
WHERE C.CustomerID > 18072
AND C.CustomerID <= 24096;
```

```
/* FRAGMENTO 5: CENTRO */
SELECT SOD.*
FROM Sales.SalesOrderDetail AS SOD
INNER JOIN Sales.SalesOrderHeader AS SOH
    ON SOD.SalesOrderID = SOH.SalesOrderID
```

```
INNER JOIN Sales.Customer AS C
  ON SOH.CustomerID = C.CustomerID
WHERE C.CustomerID > 24096
AND C.CustomerID <= 30118;
```

Validar el diseño de la fragmentación para verificar que cumpla las 3 reglas de fragmentación

Las reglas deben cumplirse para las relaciones fragmentadas. La demostración principal se realiza sobre Sales.Customer y posteriormente se comprueba que las fragmentaciones derivadas de SalesOrderHeader y SalesOrderDetail preserven las mismas propiedades.

Completitud

Toda tupla de la relación original debe pertenecer al menos a un fragmento. En Customer, los cinco intervalos cubren de 1 a 30118 sin dejar intervalos fuera.

$$\text{Customer} = C1 \cup C2 \cup C3 \cup C4 \cup C5$$

La comprobación SQL consiste en comparar COUNT(*) de la relación original contra el total obtenido por los cinco fragmentos. Para Customer, el total original observado fue 19820 filas.

Reconstrucción

Debe ser posible recuperar la relación original a partir de sus fragmentos. Para una fragmentación horizontal se utiliza la unión.

$$\text{Customer} = C1 \cup C2 \cup C3 \cup C4 \cup C5$$

El total reconstruido debe coincidir con el total original. La misma idea se aplica a SalesOrderHeader y SalesOrderDetail usando sus cinco fragmentos derivados.

Disyunción

Una tupla no debe pertenecer simultáneamente a dos fragmentos diferentes. Los límites se definieron con operadores complementarios, por ejemplo, C1 termina en CustomerID ≤ 6024 y C2 inicia con CustomerID > 6024 .

$$C_i \cap C_j = \emptyset, \text{ para } i \neq j$$

Comprobación para SalesOrderHeader

Los cinco fragmentos derivados son:

$$SOH_1, SOH_2, SOH_3, SOH_4, SOH_5$$

Regla 1. Completitud

Para que exista completitud, todas las órdenes de SalesOrderHeader deben quedar asignadas a alguno de los cinco fragmentos derivados.

En nuestro diseño los clientes están distribuidos desde CustomerID = 1 hasta CustomerID = 30118.

Compleitud de SalesOrderHeader: se verificó que todas las tuplas de SalesOrderHeader tengan correspondencia con alguno de los fragmentos de Customer. La consulta busca órdenes cuyo CustomerID no se encuentre dentro de la fragmentación establecida. Si el resultado es cero, se comprueba que ninguna orden quedó fuera y, por lo tanto, se cumple la propiedad de completitud.

Regla 2. Reconstrucción

Ahora debemos comprobar que:

$$\boxed{SalesOrderHeader = SOH_1 \cup SOH_2 \cup SOH_3 \cup SOH_4 \cup SOH_5}$$

Reconstrucción de SalesOrderHeader: se realizó la unión de los cinco fragmentos derivados utilizando UNION ALL. El número de registros obtenido debe ser igual al número de registros de la relación original SalesOrderHeader. Debido a que los fragmentos únicamente seleccionan filas de la relación original y no modifican sus atributos, la igualdad de registros, junto con la comprobación de disyunción, permite verificar que la relación puede reconstruirse a partir de sus fragmentos.

Regla 3. Disyunción

Ahora comprobamos que una misma orden no aparezca en dos regiones.

La llave que podemos utilizar es:

$$SalesOrderID$$

Si devuelve cero filas:

$$SOH_i \cap SOH_j = \emptyset, \quad i \neq j$$

Se verificó que cada SalesOrderID aparezca como máximo en un fragmento. Debido a que cada orden pertenece a un único cliente y los rangos de CustomerID no se traslapan, una orden no puede pertenecer simultáneamente a dos regiones. Si la consulta devuelve cero filas, se cumple la propiedad de disyunción.

Comprobación para SalesOrderDetail

Ahora hacemos exactamente el mismo razonamiento, pero un nivel más abajo:

$$Customer_i \rightarrow SOH_i \rightarrow SOD_i$$

La fragmentación derivada es:

$$SOD_i = SalesOrderDetail \bowtie_{SOD.SalesOrderID=SOH_i.SalesOrderID} SOH_i$$

Regla 1. Completitud

Debemos verificar que cada detalle de orden pueda llegar hasta un cliente perteneciente a alguno de nuestros fragmentos.

Si obtenemos 0, significa que todos los detalles pueden asignarse a alguno de los cinco fragmentos.

Complejidad de SalesOrderDetail: se verificó que cada detalle de orden tenga correspondencia con una orden de SalesOrderHeader y que dicha orden corresponda a un cliente incluido en alguno de los cinco fragmentos. Si la consulta devuelve cero registros sin fragmentar, se comprueba la propiedad de completitud.

Regla 2. Reconstrucción

Tenemos que poder cumplir:

$$\text{SalesOrderDetail} = SOD_1 \cup SOD_2 \cup SOD_3 \cup SOD_4 \cup SOD_5$$

Se realizó la unión de los cinco fragmentos derivados mediante UNION ALL. El número total de filas reconstruidas debe coincidir con el número de filas de la relación original SalesOrderDetail. De esta forma se verifica que los detalles de las órdenes pueden recuperarse a partir de los cinco fragmentos.

Regla 3. Disyunción

Aquí debemos tener un poco de cuidado. Una orden tiene varios detalles, por lo que no debemos comprobar únicamente SalesOrderID.

Para identificar una fila de detalle utilizamos:

$$(\text{SalesOrderID}, \text{SalesOrderDetailID})$$

Por lo tanto:

$$SOD_i \cap SOD_j = \emptyset, \quad i \neq j$$

Se verificó que cada detalle, identificado mediante SalesOrderID y SalesOrderDetailID, pertenezca únicamente a uno de los cinco fragmentos. Como cada orden pertenece a un solo cliente y los rangos de CustomerID no se traslapan, los detalles asociados a una orden tampoco pueden aparecer en dos regiones diferentes. Si la consulta devuelve cero filas, se comprueba la propiedad de disyunción.